

CAMtoBEh

*Real-Time Behavioral Analysis and Mobile Device Detection
in Smart Classrooms*

Students: Mattia Cainarca, Riccardo Cristallo, Mattia Verdolin

Supervisor: Mirko Gelsomini

Co-supervisor: Francesca Fusco

Client: SUPSI

Course: Master of Science in Engineering

Module: TSM_IoT_26

Year: 2026

1. Introduction

CAMtoBEh is a real-time behavioral analysis system for indoor environments, designed to process audio-video streams and produce structured information about the behavior of people present in a room. The system focuses on smart classroom context and on detecting digital distractions related to smartphone use.

The application combines video acquisition, person detection, object recognition, tracking, and a simplified estimation of head orientation, with two main operating modes: *phone detection* and *attention detection*.

1.1 Context

The reference context is that of smart classrooms, where the system must observe multi-user dynamics, detect the presence of personal devices, and produce real-time behavioral information useful to teachers or supervisors.

1.2 Objectives

The project objectives are:

- acquire video streams from an IP camera or from a webcam compatible with OpenCV;
- detect people and smartphones in the scene;
- estimate the attention state based on head orientation;
- display annotated results in real time;
- provide concise session or time-interval summaries.

1.3 Use Case

In the prototype, the system performs the following operations:

- detects people present in the scene via object detection and visual tracking;
- identifies the mobile phone as the object of interest within the frame;
- evaluates the association between a person and a phone through spatial overlap of bounding boxes;
- classifies the subject as *attentive* or *distracted* in the dedicated mode, using head orientation as an approximated indicator of attention direction;
- generates visual output and summary popups during or at the end of execution.

The system supports two configurable attention sub-criteria:

- **Lecture mode:** the subject is considered distracted if they look downward;
- **Exercise mode:** the subject is considered distracted if they look upward.

2. System Architecture

This chapter describes both the hardware setup and the software design of CAMtoBEh, covering the physical components used for video acquisition and networking, the technologies adopted for inference and control, and the modular structure of the application.

2.1 Hardware Architecture

The hardware architecture consists of a TP-Link Tapo C225 camera, a Zyxel WAP3205 v2 router/access point, and a dedicated processing computer for AI inference.

Component	Function
TP-Link Tapo C225	Primary video/audio acquisition sensor
Zyxel WAP3205 v2	Local network management
Processing PC	Execution of AI models and application logic

The camera streams video over RTSP on the local network managed by the Zyxel access point. The processing PC connects to the same network and pulls the stream for real-time inference.

2.2 Software Stack

The project is developed in Python and makes use of computer vision libraries and pretrained models for acquisition, inference, visualization, and application control.

Technology	Role
Python	General orchestration, threading, control logic, and GUI
OpenCV	Stream acquisition, graphical rendering, video annotations, and vision functions such as solvePnP
MediaPipe Object Detector	Detection of people and smartphones in phone detection mode
MediaPipe Face Landmarker	Extraction of facial landmarks for face pitch estimation
YOLOv8 Pose	Person tracking and body keypoints in attention mode
Tkinter	Control panel and summary popups

2.3 Software Modules

The software structure is modular and divides the system into components with independent responsibilities, facilitating integration, maintenance, and real-time execution.

- **RTSPReceiver:** acquires the video stream and retains only the most recent frames using a minimal buffer (`deque(maxlen=2)`);
- **Processor:** executes the phone detection mode using MediaPipe Object Detector;
- **AttentionProcessor:** executes the attention detection mode using YOLOv8 Pose and Face Landmarker;
- **Display:** renders the annotated frame and summary information in phone detection mode;
- **ControlPanel:** allows the user to change modes and parameters at runtime;
- **main:** coordinates all modules, handles mode switches, and manages periodic summaries.

2.4 Data Flow

1. The video stream is acquired by RTSPReceiver.
2. The most recent frame is forwarded to the active inference module.
3. The module processes the frame and produces annotations and statistics.
4. The result is displayed in the main window.
5. The system updates popups, counters, or session summaries based on the active mode.

3. Detection Pipelines

CAMtoBEh implements two independent inference pipelines that share the same video input but differ in their processing logic, models, and output semantics. The active pipeline can be switched at runtime without restarting the application.

3.1 Phone Detection

The phone detection mode uses the MediaPipe Object Detector with the `efficientdet_lite2.tflite` model, which is automatically downloaded if not present locally.

3.1.1 Processing Pipeline

1. Acquisition of the current frame.
2. Conversion from BGR to RGB for MediaPipe compatibility.
3. Asynchronous inference with `detect_async()`.
4. Filtering of detections based on the configured confidence threshold.
5. Separation between objects labeled person and cell phone.
6. Bounding box overlap check to identify people associated with a phone.
7. Frame annotations with green boxes for people and red boxes for people holding a phone.

3.1.2 Output

For each frame, the following are returned:

- annotated frame;
- number of detected people;
- number of people associated with a phone.

In the main loop, the system also updates cumulative statistics such as the maximum number of people in the scene and the maximum number of people with a phone during the observation interval.

3.2 Attention Detection

The attention detection mode uses YOLOv8 Pose for person tracking and MediaPipe Face Landmarker for face orientation estimation.

3.2.1 Processing Pipeline

1. YOLOv8 Pose detects people and body keypoints.
2. Nose and eye keypoints are used to build a face ROI via `_get_face_crop()`.
3. The ROI is resized to 224×224 pixels.
4. Face Landmarker extracts facial landmarks from the face crop.
5. Six landmarks are mapped onto a simplified 3D face model defined in the code.
6. `cv2.solvePnP()` and `cv2.RQDecomp3x3()` are used to derive the pitch angle.
7. The angle is compared against predefined thresholds to classify the subject as attentive or distracted.

3.2.2 Operating Sub-modes

The classification uses two configurations, selectable from the control panel:

- **MODE_DOWN_DISTRACTED:** the subject is distracted if looking downward, intended for frontal lecture scenarios;

- **MODE_UP_DISTRACTED:** the subject is distracted if looking upward, intended for exercise or screen-based sessions.

3.2.3 Session Metrics

The module progressively updates:

- number of checks performed;
- percentage of attentive subjects in the current frame;
- overall session average;
- timestamped log of the most recently observed samples.

4. User Interface

The graphical interface is built with Tkinter and serves as the application control point during execution. It operates alongside the main OpenCV display window and allows parameter adjustments without interrupting the inference loop.

4.1 Control Panel

The ControlPanel allows the user to:

- select either *phone* or *attention* mode;
- modify the smartphone detector confidence threshold;
- choose the distraction criterion for the attention mode;
- enable or disable summary popups.

4.2 On-Screen Information

During execution, the main OpenCV window displays:

- bounding boxes and labels for detected objects;
- concise counts of people and phones;
- subject attention state with a traffic-light color scheme;
- session average and number of samples in attention mode.

4.3 Keyboard Shortcuts

In addition to the graphical panel, the following keyboard shortcuts are available at runtime:

- **SPACE**: toggle between Phone Detection and Attention Detection mode;
- **M**: switch attention criterion between lecture and exercise mode;
- **P**: reopen or bring the control panel to the foreground;
- **Q**: close the application.

5. Installation and Operation

This chapter covers the full procedure required to set up and run CAMtoBEh, from environment preparation and configuration to startup and runtime verification.

5.1 Requirements

The following elements are required to run the system:

- a processing computer running Windows, Linux, or macOS;
- Python version between 3.10 and 3.12;
- an IP camera with an RTSP stream accessible on the local network (e.g. TP-Link Tapo C225), or alternatively a webcam compatible with OpenCV;
- a local network connection shared between the video source and the inference computer;
- the required software libraries: opencv-python, mediapipe, numpy, ultralytics, python-dotenv;
- the tkinter GUI module, already included in the standard Python distribution on Windows and macOS; on Linux it may require separate installation.

5.2 Environment Setup

The setup procedure involves creating a dedicated virtual environment to isolate project dependencies.

1. Open a terminal in the project folder.
2. Create a virtual environment:

```
python -m venv venv
```

3. Activate the virtual environment (Linux/macOS):

```
source venv/bin/activate
```

4. On Windows:

```
venv\Scripts\activate
```

5. Install the project dependencies:

```
pip install -r requirements.txt
```

The requirements.txt file must include at least opencv-python, mediapipe, ultralytics, numpy, and python-dotenv.

5.3 Configuration

5.3.1 .env File

The main system configuration is centralized in the .env file, read automatically via the python-dotenv library. Create it from the provided template:

```
cp .env.example .env
```

On Windows, manually create a .env file in the project root. The fundamental parameter to configure before startup is:

```
RTSP_URL=rtsp://USER:PASSWORD@IP_CAMERA:554/stream2
```

For the Tapo C225, the correct username, password, IP address, port, and stream path must be provided.

5.3.2 Available Parameters

Additional configurable parameters in the .env file are:

- WINDOW_NAME: name of the OpenCV window;
- PHONE_MODEL_PATH and PHONE_MODEL_URL: local path and URL for the EfficientDet Lite2 model;
- PHONE_CONF_THRESHOLD: initial confidence threshold for the smartphone detector;
- YOLO_MODEL_PATH and YOLO_MODEL_URL: path and URL for the YOLOv8n-pose model;
- YOLO_CONF_THRESHOLD: initial threshold for pose detection;
- FACE_LANDMARKER_PATH and FACE_LANDMARKER_URL: path and URL for the MediaPipe Face Landmarker model;
- DOWN_THRESHOLD: pitch threshold for lecture mode;
- UP_THRESHOLD: pitch threshold for exercise mode;
- MIN_PROCESS_INTERVAL: minimum interval between two consecutive inferences in attention mode.

5.3.3 Automatic Model Download

On first launch, if the model files are not present locally, the program automatically downloads and saves them to the paths specified in .env:

- a MediaPipe EfficientDet Lite2 model for detecting people and phones;
- a YOLOv8n-pose model for body keypoint extraction;
- a MediaPipe Face Landmarker model for facial landmark analysis and head orientation estimation.
- This step occurs only once, unless the downloaded files are manually removed.

5.4 Startup and Verification

1. Connect the IP camera and the processing computer to the same local network.
2. Verify that the RTSP stream is reachable at the address defined in RTSP_URL. Alternatively, configure a local OpenCV-compatible source for webcam use.
3. Launch the main application:

```
python main.py
```

4. Wait for the video source to load and the models to initialize.
5. Verify that the frame appears in the main OpenCV window and that the Tkinter control panel opens.
6. Select the desired operating mode (*Phone Detection* or *Attention Detection*) and adjust parameters as needed.

After startup, correct operation can be confirmed by checking that:

- the video frame updates in real time in the main window;
- bounding boxes appear around people and phones in phone detection mode;
- labels classifying subjects as attentive or distracted appear in attention mode;
- the control panel responds to mode changes and threshold adjustments;

- summary popups are displayed when triggered by the execution logic.

5.5 Operational Notes

For more stable results during testing:

- use a local network with good signal quality between the camera and computer;
- maintain adequate scene illumination;
- position the camera to keep the face and upper body of subjects visible;
- avoid prolonged occlusion of the face or phone;
- adjust the confidence threshold based on the subject's distance from the camera.

6. Evaluation and Outlook

6.1 Technical Issues and Adopted Solutions

Problem	Implemented Solution	Technical Detail
Video stream latency	Minimal frame buffer	RTSPReceiver uses deque(maxlen=2) to prioritize the most recent frame
Computational load in attention mode	Processing frequency limitation	AttentionProcessor introduces _min_process_interval = 0.12
Face not always reliably detected	Face ROI with fallback	The crop uses nose/eyes if available, otherwise the upper portion of the person bounding box
Variability in smartphone detections	Runtime-adjustable confidence	The threshold is set from the panel and propagated to the detector
Need to switch system function during demo	Distinct pipelines with controlled switch	main stops the active module and initializes the next one

6.2 Prototype Limitations

The system implements a functional prototype focused on two main use cases and presents some structural limitations:

- user identity is not yet managed as persistent biometric recognition;
- attention estimation is based on head pitch and not on precise eye tracking;
- the person-phone association is based on spatial overlap and not on full hand-object analysis;
- output is primarily visual and not yet exported to a persistent structured stream such as JSON or MQTT.

6.3 Future Developments

The most coherent developments with the current architecture are:

- export of events to JSON or MQTT;
- persistent subject tracking across frames;
- more advanced hand-object interaction analysis;
- a remote dashboard for the teacher or supervisor, consistent with the alert gateway concept;
- quantitative validation with accuracy, robustness, and latency metrics.

6.4 Conclusion

CAMtoBEh realizes a behavioral analysis prototype for smart classrooms capable of integrating video acquisition, person detection, smartphone recognition, and simplified real-time attention estimation. The modular software structure, the integration of OpenCV, MediaPipe, and YOLOv8, and the presence of a dedicated control panel make the system replicable in a laboratory setting and suitable for demonstrating the presented use case.